

GSoC 2026 Proposal

Google Summer of Code

Project for OWASP: AI Assisted Contributor Guidance for BLT Preflight



OWASP BLT

Basic Information:

1. Full Name	Savio D'souza
2. Github	https://github.com/S3DFX-CYBER
3. Gmail	
4. Linkedin	https://www.linkedin.com/in/savio-d-souza-0a1810349/
5. TryHackMe	https://tryhackme.com/p/savioudsouza8a
6. Contact	+91 XXXXXXXXXXXX
7. TimeZone	Indian Standard Time (UTC +5:30)
8. Location	XXXX, India

Project Details:

Project Title: AI Assisted Contribution Guidance for BLT-Preflight

Project Size: Large

Project Duration: 350 hours

Project Mentors: Donnie Brown, Abhishek Kumar. Kunal Kashyap

Table of Contents

1. Project Overview	3
2. Project Goals & Deliverables	5
3. Terminologies & Concepts	8
4. Execution Planning/Timeline.....	21
5. LLM Usage Considerations.....	30
6. Testing & Evaluation	32
7. Risks & Mitigation	34
8. About Me	37
9. Motivation & Post GSoC	38
10. References	43

Project Overview:

OWASP BLT (Bug Logger Tool) has introduced a new feature called BLT Preflight. It is designed to provide security intent and risk guidance before contributors submit code to prevent common mistakes. It reviews and evaluates security context through issue labels, repository metadata and historical patterns for operational readiness, security risks, and production-impacting changes before they ship. Inspired by the surge of AI-generated PRs, Preflight helps teams maintain high standards by adding an automated layer of critical thinking and risk detection to every change.

While BLT's primary stack is Cloudflare-native, BLT-Preflight operates as a developer-local tool and GitHub Actions sidecar not as a BLT web service. This is why FastAPI + Docker is appropriate here: the pre-commit hook and VS Code extension both require localhost access, which Cloudflare Workers cannot provide.

The Problem Foundation BLT-Preflight currently in an early stage, it helps in catch common mistakes through issue labels and file patterns etc. There is no intelligent mechanism that can help contributors understand risks and security expectations before they submit their code. For a first-time BLT contributor, there is no signal before submitting a PR about whether their change touches a security-sensitive area. For

maintainers, every PR requires manual security review with no automated baseline.

Hence, to solve this Problem, I propose Project AI Assisted Contribution Guidance for BLT-Preflight, an intelligent layer with RAG-powered advisory capabilities delivered across three tiers meeting contributors where they are, at every stage of development.

The three tiers work together as a defense-in-depth guidance pipeline:

Tier	Trigger	Delivery
Tier-1 IDE	File save / on-type in VS Code	Inline hints and security diagnostics in the editor
Tier-2 Pre-Commit	git commit on the local machine	Blocking or advisory check before the commit is recorded
Tier-3 Pull Request	PR opened / updated on GitHub	Structured advisory comment posted by GitHub Actions

Minimal RAG scope for this project is defined as: a single vector store of curated OWASP documentation (Top 10, Auth Cheat Sheet, API Security), embedding-based retrieval returning the top 3 relevant chunks per query, and a single LLM call per PR event generating a structured advisory no multi-turn conversation, no fine-tuning, no real-time web retrieval.

The system is cross-platform by default: a single Docker image runs the local FastAPI advisory server identically on Windows, macOS, and Linux, with no manual dependency installation required

The system aims to provide maintainers and contributors with an advisory system for identifying and understanding security intent and risks before submitting code, with recommended remediation actions, without blocking contributions.

This proposal builds on discussions and feedback from OWASP BLT maintainers and mentors, which I have refined into a detailed implementation plan.

Project Deliverables for GSoC:

- Develop a Risk Classification (heuristic signals based) Engine with AI assisted advisory generation

- Create contextual remediation guidance using RAG-powered retrieval over curated OWASP documentation, scoped to top 3 chunk retrieval per PR event with a single structured LLM call
- VS Code Extension (Tier 1): A lightweight VS Code extension that calls the local advisory server on file-save and surfaces inline security hints as VS Code Diagnostics no blocking, no enforcement
- Pre-Commit Hook (Tier 2): A pre-commit framework hook that runs local signal extraction and advisory generation before every git commit, giving contributors security feedback before code ever leaves their machine
- Cross-Platform Docker Setup: A single docker-compose up command that starts the local advisory server identically on Windows, macOS, and Linux no manual Python or dependency setup required
- Deployment and configuration documentation
- Testing on real repositories and Production ready PR advisory and contributor guidance system for BLT-Preflight

Project Goals:

The Project's main goal is to provide clear and contextual contribution guidance for newcomers and contributors, to detect common mistakes related to security risks and improve pull requests quality , while

supporting contributors in aligning their work with the project expectations.

Other goals of this project are to explain the contributors why it matters from a security perspective , what changes are required or suggested safe alternatives to improve the code from a security perspective.

POST-GSOC:

- **ML-based Risk Classification:** A logistic regression classifier trained on accumulated PR metadata to predict risk levels. The heuristic scoring engine built during GSoC will serve as the foundation for this.
- **Explicit Contributor Intent Capture:** A lightweight PR checklist or form that allows contributors to declare their intent before submitting, giving the system additional context beyond passive signal detection.
- **Extended Maintainer Dashboard:** Expanding the minimal dashboard built during GSoC into a full view with trends, per-contributor advisory history, and team-level security insights.

- Expanded OWASP Knowledge: Adding more OWASP cheat sheets and project-specific documentation as new guidance is published, keeping the advisory system current
- VS Code Extension Marketplace Listing: A full public listing on the VS Code Marketplace with ratings, reviews, and auto-update support (the .vsix package produced during GSoC will be available for manual installation, with Marketplace publication as a post-GSoC polish step if not completed in Week 9).
- Multi-IDE Support: JetBrains plugin (IntelliJ, PyCharm) and Neovim LSP integration based on the same FastAPI advisory API contract.

Few Terminologies:

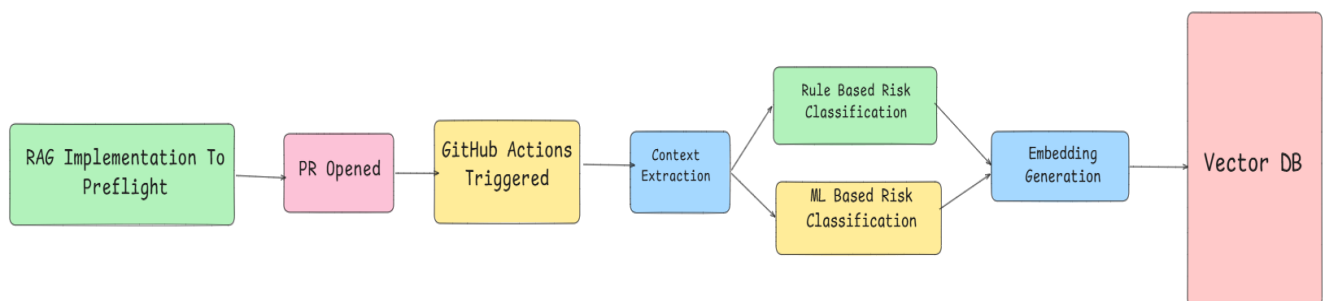
- Why RAG
 - Without RAG assistance, the LLM may hallucinate advice related to security and risks which may lead to improper guidance for the contributor. With RAG, it will retrieve information from a trusted and known source and

keeps itself updated with latest information and knowledge.

NOTE: RAG is used for only contextual guidance retrieval not implementing risk scores, decision making, blocking prs.

- Why Embeddings

- Embeddings are one of the most important part of an RAG based model as they allow the system to retrieve relevant security guidance even when the contributor's context doesn't exactly match the keywords.



Predicted Implementation:

When a PR is opened , Github Action is triggered which then extracts contextual signals like issue labels, changed files and repository metadata.

Then, the system performs a hybrid risk classification assessment using rule-based patterns. The extracted context is then used as a semantic query to retrieve related OWASP based sources for guidance from a vector store via embeddings. Which then is used to generate the explainable and concise advisory remediation guidance for the contributor.

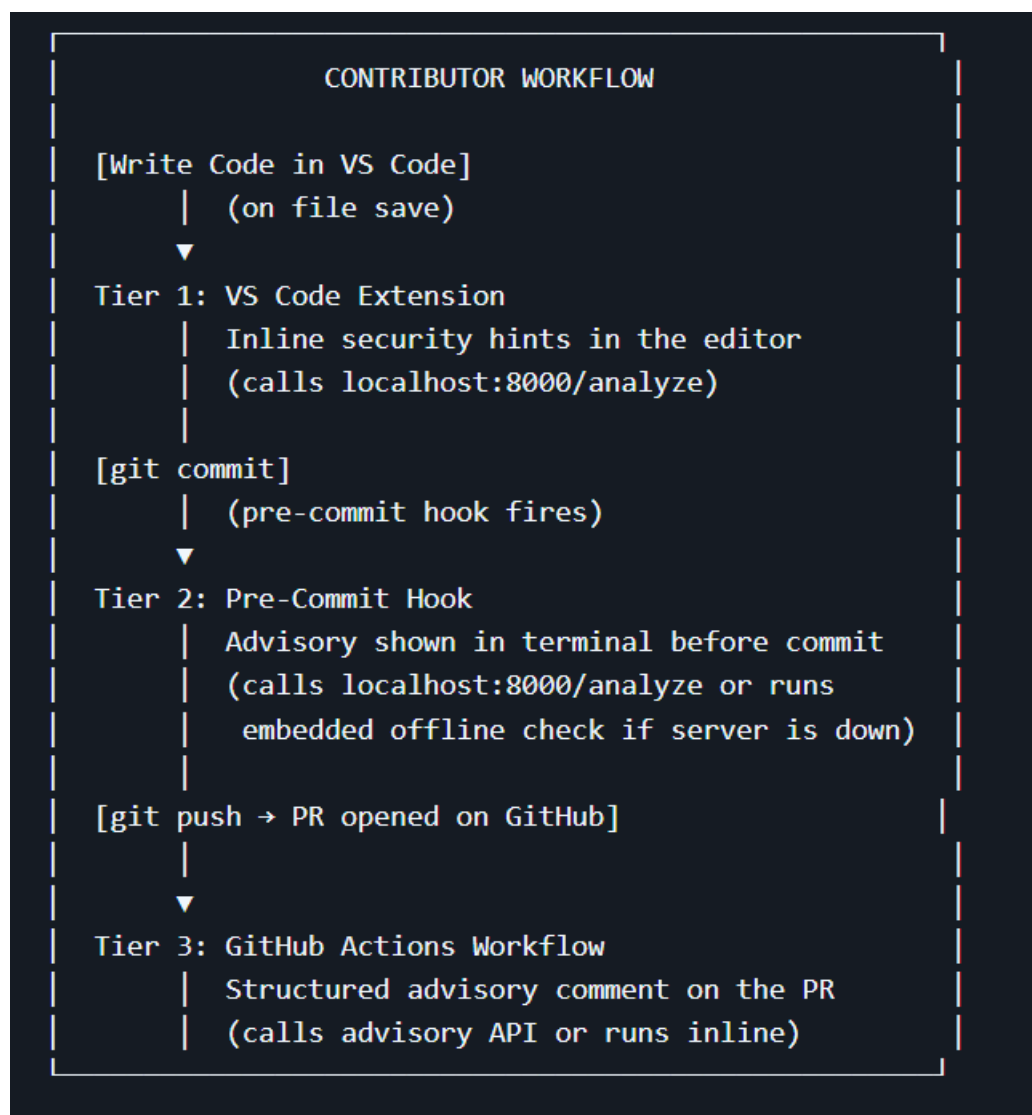
Hybrid Retrieval Strategy:

The RAG system doesn't just rely on vector search. It uses a three-step hybrid approach to ensure both precision and coverage:

1. Rule-based Signal Mapping: Before any vector search runs, the system checks the PR for known signals and maps them directly to OWASP categories for example:
 - A PR modifying auth.py, login.js or session.py maps to OWASP A07
 - A PR touching database models or query files maps to OWASP A03
 - A PR adding new API endpoints maps to OWASP A01
2. Semantic Vector Search: After the rule-based mapping, the full PR context (changed files, labels, commit message etc) is used as a query to search the OWASP documentation vector store using embeddings. This catches risks the rule-based layer may miss.

3. Result Merging: After both steps, results from both steps are combined and deduplicated before being passed to the LLM. The LLM then generates the final advisory based on this merged context.

- Three-tier Local First Architecture:
 - The advisory system is designed as a local-first pipeline so that contributors receive guidance at the earliest possible moment before code is even committed rather than only after a PR is opened.



All three tiers share the same FastAPI advisory server (preflight-server), which runs locally via Docker or natively. This single-server design means the exact same risk scoring logic and RAG retrieval is used whether the check is triggered from the editor, the terminal, or a GitHub Actions runner ensuring consistent guidance across the full workflow.

- Cross-Platform Setup:

The advisory server ships as a Docker container so it runs identically on Windows, macOS, and Linux without requiring contributors to manage Python environments, pip conflicts, or OS-specific paths.

One-Command Setup:

`docker-compose up -d preflight-server`

This starts the advisory server at localhost:8000. The VS Code extension and pre-commit hook both point to this endpoint. For air-gapped or privacy-sensitive environments, the Ollama fallback model runs in a second container defined in the same docker-compose.yml, keeping all inference on-premises.

Native Python install (alternative): For contributors who prefer not to use Docker, a setup.sh / setup.bat bootstrap script handles virtual environment creation, dependency installation, and the pre-built ChromaDB index download in a single command. Both paths are documented and tested on Ubuntu 22.04, macOS 14, and Windows 11.

- VS Code Extension Architecture:

The VS Code extension (blt-preflight-vscode) is a lightweight TypeScript extension that integrates security guidance directly into the editor without interrupting the coding flow.

How it works:

1. On file save, the extension reads the workspace's changed files since the last commit using `git diff --name-only HEAD`. In a repository without any commits (newly initialized), it falls back to listing all tracked files via `git ls-files` so the extension works from the very first file saves.
2. It posts a JSON payload containing the changed file paths and the active file's name to `localhost:8000/analyze`, with a 500ms debounce so rapid consecutive saves do not generate multiple requests.
3. The advisory server performs signal extraction and RAG retrieval and returns a structured JSON advisory.
4. The extension maps each advisory item to a VS Code Diagnostic object with severity (Error for HIGH, Warning for MEDIUM, Information for LOW) and displays it inline in the Problems panel and as editor gutter icons.
5. Clicking a diagnostic opens a Quick Fix panel with the full advisory explanation and OWASP references. If the advisory server does not respond within 2 seconds, the extension silently skips and tries again on the next save it never blocks the editor.

Key design constraints:

- The extension communicates only with localhost no external network calls from the editor.
- Diagnostics are advisory-only: they do not prevent saving or building.
- The server URL is configurable (preflight.serverUrl in VS Code settings) so teams can point the extension at a shared team advisory server if desired.
- If the advisory server is not running, the extension silently disables itself with a status bar indicator it never blocks the editor.

Extension activation events:

onLanguage:python, onLanguage:javascript, onLanguage:typescript, onLanguage:java limiting activation to relevant file types to minimize IDE overhead.

○ **Pre-Commit Hook Integration**

The pre-commit hook integrates BLT Preflight into the standard git commit workflow using the [pre-commit](#) framework, which manages hook installation and updates declaratively.

Installation (contributor runs once):

pip install pre-commit

pre-commit install

After this, every git commit automatically triggers the Preflight check. The hook configuration is committed to the repository as `.pre-commit-config.yaml`:

```
repos:
  - repo: https://github.com/OWASP/BLT
    rev: v1.0.0
    hooks:
      - id: blt-preflight
        name: BLT Preflight Security Advisory
        language: python
        entry: preflight-hook
        types: [python, javascript, typescript]
        stages: [commit]
```

- **Hook behaviour:**

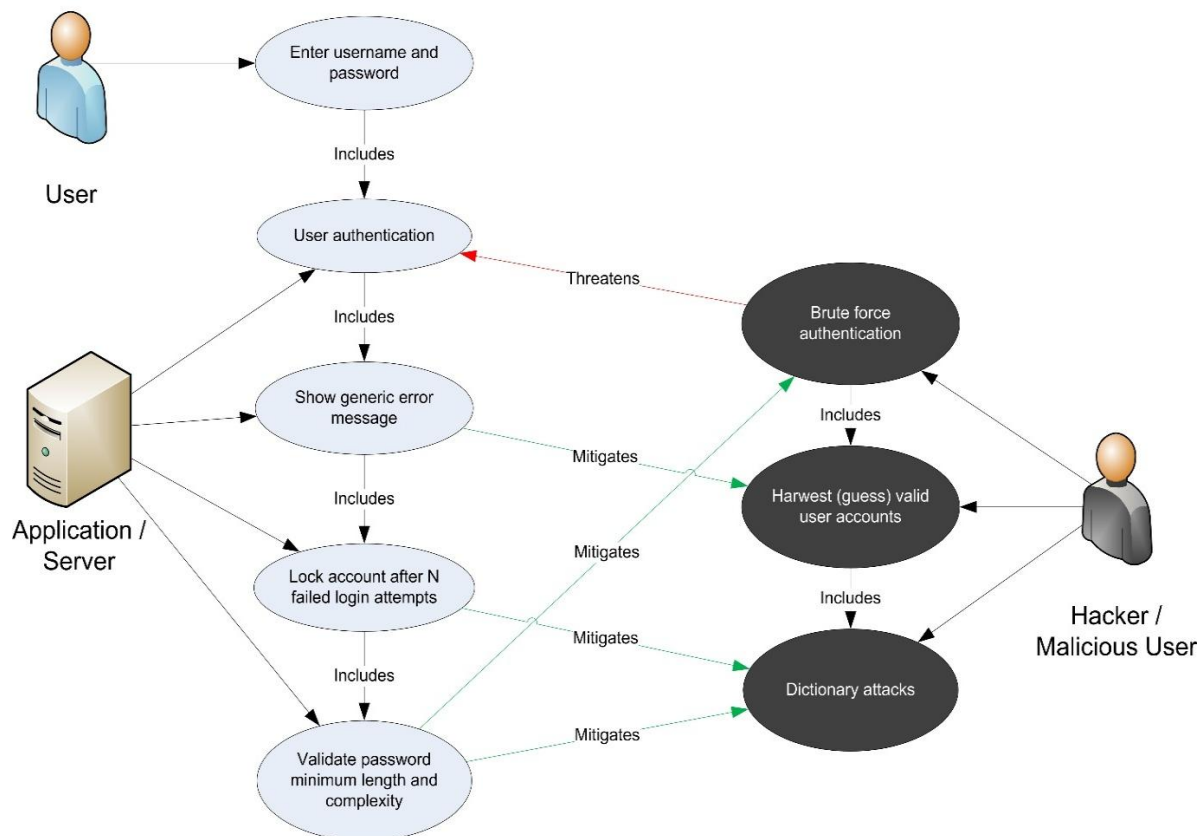
- The hook calls the local advisory server (localhost:8000/analyze) with the staged file list (***git diff --cached --name-only***). Note: the pre-commit hook intentionally uses staged files rather than the VS Code extension's ***git diff --name-only HEAD*** because at commit time the staged set is the precise change being recorded; the extension checks against HEAD to surface guidance as early as possible during editing.
- If the server is unavailable, the hook runs an **embedded offline check** using rule-based signal mapping (no LLM, no embeddings). This offline path always exits 0, so an unavailable server never blocks a commit.
- When the server is available and returns a HIGH severity finding, the hook prints a clear warning to the terminal and **exits 1** (blocking the commit) by default. This behaviour is configurable via `.preflight.yaml`:
preflight:

```
preflight:
  hook_mode: advisory      # 'advisory' (warn only) or 'blocking' (exit 1 on HIGH)
  offline_fallback: true   # use rule-based fallback if server is unavailable
  minimum_severity: MEDIUM # suppress LOW advisories in hook output
```

- For MEDIUM and LOW, the hook prints the advisory and exits 0 (commit proceeds).

○ Threat Modelling

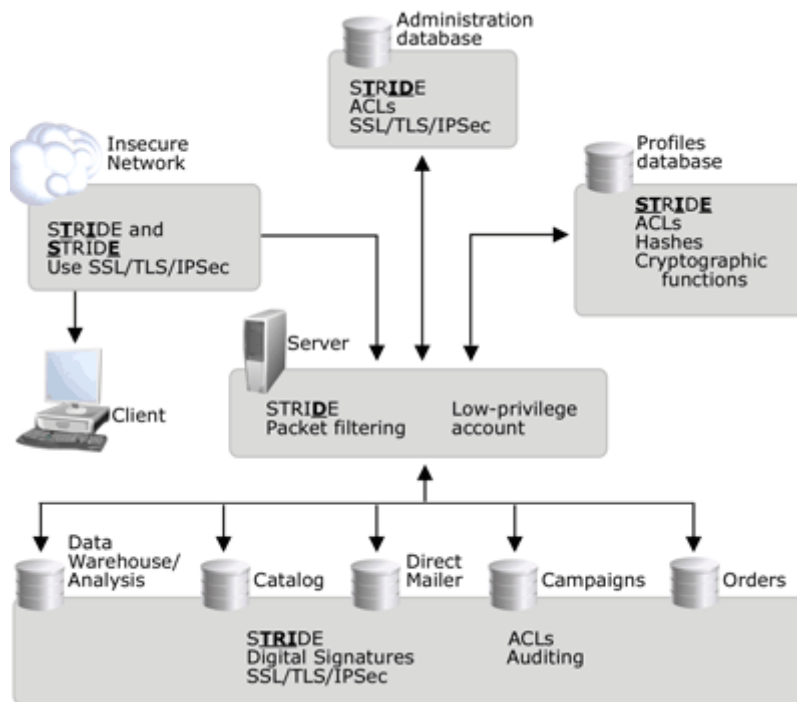
- Threat Modelling is a structured process for identifying and mitigating security threats and risks to an application, system or a network in the early development cycle.



○ STRIDE

1. Spoofing : A threat that is aimed to gain unauthorized access of another user's credentials such as username and password.

2. Tampering : A threat that is intended to maliciously modify data such as records or any important data in a database, which can lead unauthorized access to sensitive information and data corruption.
 3. Repudiation : A threat that is aimed to perform illegal operations over a system that lacks the ability to trace operations.
 4. Information Disclosure: A threat that exposes information related to stack or credentials visible to users, which can be a big advantage for attackers.
 5. DoS : A threat action that attempts to overwhelm the server temporarily and makes it unavailable or unusable.
 6. Elevation of privilege : A threat that is intended to gain privileged access to information in order to gain unauthorized access or compromise a system.
- Application of STRIDE:
- STRIDE will be used to identify and mitigate the risks related to identity, data integrity, and confidentiality across the Preflight pipeline.
 - Tampering : A malicious contributor could craft commit messages or labels designed to manipulate the heuristic risk score.
Mitigation: signals are sanitized and weighted, never trusted as raw input.
 - Information Disclosure : File names and commit messages sent to Gemini Flash could expose internal path names or proprietary logic.
Mitigation: only metadata is sent to the LLM raw file content and diffs are explicitly excluded.
 - Spoofing: The workflow must only respond to authenticated PR events.
Mitigation: GitHub's built-in GITHUB_TOKEN scoping ensures only legitimate repository events trigger the workflow.
 - Elevation of Privilege: The GitHub token posting advisory comments must be minimally scoped.
Mitigation: token restricted to pull-requests: write only, no admin access.



Tech Stack:

Component	Choice	Reason
Embedding Model	sentence-transformers (all-MiniLM-L6-v2)	It is lightweight, offline and doesn't cost for API
VectorDB	ChromaDB	Zero-config, Python-native, easier to use and beginner friendly
LLM Provider	Gemini Flash	Free and cost-effective
Framework	FastAPI	Lightweight, asynchronous, and provides clean API boundaries
VS Code Extension	TypeScript + VS Code Extension API	Native diagnostic integration, no editor restart required
Pre-Commit Hook	Python + pre-commit framework	Single-command install, cross-language support, widely adopted

Cross-Platform Runtime	Docker + Docker Compose	Identical environment on Windows, macOS, and Linux
------------------------	-------------------------	--

- **Why these choices over various alternatives:**
- all-MiniLM-L6-v2 over larger models: Produces 384-dimension embeddings and runs in under 100ms on CPU, making it viable inside a GitHub Actions runner without GPU access. Larger models like all-mpnet-base-v2 offer marginally better accuracy but are 3–4x slower, pushing latency beyond acceptable limits for a PR workflow tool.
- ChromaDB over FAISS: FAISS is index-only with no built-in persistence or metadata filtering. ChromaDB persists to disk automatically and supports metadata filters needed for hybrid retrieval. Qdrant is a strong alternative but requires a running server process, adding unnecessary infrastructure complexity at this scale.
- LLM Fallback Strategy: Primary is Gemini Flash (free tier for development and low-traffic repos). Fallback option A is GPT-4o-mini (paid, low cost per token). Fallback option B is a local model via Ollama (e.g., mistral:7b) for fully offline or privacy-sensitive deployments. Fallback is triggered on HTTP 429 or 5xx response codes and is configurable via the YAML config file. API keys are stored as GitHub Actions encrypted secrets and never hardcoded.
- Gemini Flash is chosen as the primary LLM because it offers low latency (1–2 seconds per call), supports structured JSON output natively which is required for the advisory schema, and provides a generous free tier suitable for development and low-traffic repositories.

- VS Code Extension API over LSP: The Language Server Protocol would be overkill for this use case. A thin VS Code extension that calls the local FastAPI server on-save and maps the JSON advisory response to VS Code Diagnostic objects is faster to ship and easier to maintain. The extension communicates only with localhost no external network calls from the editor.
- Pre-commit framework over raw Git hooks:
Raw .git/hooks scripts are machine-local and not committed to the repository. The pre-commit framework manages hook installation, versioning, and multi-language support declaratively via a .pre-commit-config.yaml file that is committed alongside the code, ensuring every contributor gets the same hooks automatically on pre-commit install.
- Docker over native install instructions: Native installation requires matching Python versions, pip environments, and OS-specific paths across Windows, macOS, and Linux. A single docker-compose up removes all of this friction. The advisory server container exposes fixed localhost:8000 endpoint consumed by both the VS Code extension and the pre-commit hook, keeping the cross-platform contract simple and stable.
- **Data Privacy and Security:**
The advisory system sends only PR metadata to the external LLM specifically : changed file names, issue labels, and commit messages. Raw file content and code diffs are explicitly excluded from all LLM prompts to prevent sensitive code exposure to third-party APIs. API keys are stored as GitHub Actions encrypted secrets. Repository maintainers who require full data privacy can configure the system to use a self-hosted Ollama model, keeping all data on-premises. Gemini's data retention policy is available at <https://ai.google.dev/gemini-api/terms>.

Advisory Log Storage: Each PR event writes a single structured record to a local SQLite database (preflight.db). The schema is:

Field	Type	Description
pr_number	Integer	Github PR number
timestamp	DateTime	UTC time of advisory generation
severity	Text	HIGH/MEDIUM/LOW
owasp_category	Text	e.g. A07:2021
dismissed	Boolean	Whether maintainer dismissed the advisory
inapplicable_label	Boolean	Whether <i>preflight-inapplicable</i> label was applied

The log explicitly does not store: contributor identity, file content, diffs, commit messages, or any LLM prompt/response text. The database is written to the repository host's local filesystem and is accessible only to maintainers with repository access. Default retention is 90 days, configurable via .preflight.yaml:

```
yaml
preflight:
  log_retention_days: 90    # set to 0 to disable logging entirely
  log_path: .preflight/preflight.db
```

The tradeoff made here is minimal stored data over richer analytics contributor-level tracking (e.g. per-author advisory history) is intentionally excluded from the GSoC scope to avoid creating a surveillance mechanism; this can be opt-in post-GSoC via the Extended Maintainer Dashboard.

- **Vector Store Persistence Strategy:**

Since GitHub Actions runners are stateless, the vector index cannot be rebuilt on every PR event this would eventually be slow and expensive. The persistence strategy is:

1. During community bonding, the OWASP doc is chunked, embedded, and stored as a pre-built ChromaDB index committed to the repository as a build artifact.
2. Each GitHub Actions run loads this pre-built index as read-only. No re-embedding occurs during normal PR processing.
3. when new OWASP documentation is added (Week 9), a maintenance script re-ingests only new documents using ChromaDB's *collection.add()* method and the updated artifact is committed.

- **Execution of Plan:**

- Community bonding period (April 30th to May 25th)
 - Conduct discussions or a slack huddle with the mentors and community to establish clear communication channels and align on project expectations.
 - Interaction with the mentors to clarify any doubts or technical design decisions if there are any. And confirm the final scope for BLT Preflight.
 - As the community bonding period consists almost 3-4 weeks i.e 26 days , we can setup the requirements needed for implementing Preflight RAG model and necessary dependencies.
 - Understanding and the architecture and technical designs of the repository

- Clone and run locally BLT Preflight and document the system's current structure and extension surfaces.
 - Small and Minimal RAG working demo based on OWASP related documentation provided to the mentors
 - Explore the VS Code Extension API and pre-commit framework documentation; prototype a minimal "hello world" extension that calls a local HTTP endpoint and surfaces a diagnostic
 - Finalize and enhance the timeline for coding period timeline based on technical findings from the prototype.
 - Prepare and design a technical document outlining
 1. System Architecture
 2. Data Ingestion and chunking strategy
 3. Embedding and retrieval approach
 4. API interface design
-
- Week 1 (May 25th to June 1st)

Coding Period Starts

 - Objective: Implement the core data ingestion and retrieval foundation, and establish the cross-platform Docker environment that all three tiers will share
 - Setup the finalised development environment and structure of the project for Preflight.(sentence-transformers, ChromaDB, FastAPI)
 - Cross-platform Docker Setup:
 1. Author Dockerfile for the advisory FastAPI server
 2. Author docker-compose.yml with the advisory server and an optional Ollama sidecar for offline LLM
 3. Verify the container starts and responds on localhost:8000 on Ubuntu, macOS and Windows

4. Write setup.sh/setup.bat for contributors who prefer native python installation.
- Implement document ingestion pipeline:
 1. Selected OWASP documentation will be loaded.(Top10, Auth Cheat Sheet, API Security etc)
 2. Parse and clean the text.
 3. Apply section-aware chunking: split at OWASP section headers such as "How to Prevent" and "Attack Scenarios" as delimiters, targeting chunk size 300-500 tokens per chunk with approximately 50 tokens of overlap between chunks to preserve context at boundaries.
 - Generate embeddings for all document chunks using sentence-transformers (all-MiniLM-L6-v2) and store them in ChromaDB.
 - Implement basic semantic retrieval using a sample contributor intent query and retrieving top relevant document chunks from ChromaDB.
 - Conduct initial retrieval testing using 4-5 examples related to contributor intent scenarios such as:
 1. "I modified the login flow"
 2. "I added a new API endpoint"
 3. "I changed how passwords are stored"
 - Share progress update and retrieval demo with the mentors at the end of the week.
- Week 2 (June 1st to June 7th)
- Objective: To transform the semantic retrieval into a structured advisory generator while validating output quality and reducing hallucination risks
 - Implement the Structured RAG Generation by connecting the retrieval layer to the LLM with a strict prompt template for the Advisory
 - Enforcing a Structured Output by implementing internal schema such as:
 1. Severity
 2. Category
 3. Explanation
 4. Recommendations

5. References

- Refining the Advisory Generation and Documentation of hallucinated control mechanism
 - Testing on 4-5 predefined scenarios like Authentication change, API endpoint addition , Secret Exposure Risk or Database Modifications etc
 - Identify the improvement areas for Week 3 with Mitigation of Risks.
- Week 3 (June 7th to June 14th)
- Objective: To integrate the RAG into Preflight pipeline and optimise performance based on Preflight
 - After successful tests, the RAG will be integrated into the Preflight pipeline
 - Ensuring stability and optimizing performance based on Preflight
 - If retrieval returns no chunks above a similarity threshold of 0.6, the system falls back to file-type-based static templates. The threshold of 0.6 is chosen based on the semantic range of all-MiniLM-L6-v2, where scores below 0.5 indicate no meaningful relationship and scores above 0.7 indicate strong relevance. 0.6 provides a conservative middle ground. This value is configurable in the YAML file.
 - Add logging for generation time and output validation.
 - Share progress update and retrieval demo with the mentors at the end of the week.
- Week 4 (June 14th to June 21st)
- Objective: To Ensure and make the system reliable and efficient with the changes
 - Optimisation of chunking size and retrieval parameters with adding embedding caching
 - Add timeout handling for LLM calls and reduce token usage

- Improve error handling and ensuring the stability of the advisory
- Share progress update with the mentors

- Week 5 (June 21st to June 28th)
 - Objective: Focus on Validating the advisory quality before midterm evaluation by mitigating hallucination risks and quality evaluation

 - Create a structured evaluation scenarios based on
 1. Authentication changes
 2. API additions
 3. Secret exposure
 4. Database modifications

 - Add a post-generation schema validation with enforcing citation only responses

 - After testing and evaluating , create an evaluation report and document the hallucination mitigation mechanisms

 - The reports and documentation will be provided to the mentors for reviewing

- Week 6 (June 28th to July 5th)
 - Midterm Evaluation Period

 - Objective: Deliver a Stable and Integrated Advisory System for Midterm Evaluation

 - Midterm Deliverables:
 1. Working RAG enhanced system for BLT Preflight
 2. Structured Advisory output generation
 3. Integrated into PR workflow
 4. Quality Evaluation results documents
 5. Hallucination Mitigation Documentation
 6. Configuration Flags

 - After Midterm Evaluation pass, Next Phase will be discussed and implemented in week 7

➤ Week 7 (July 5th to July 12th)

- Objective: Enhancing Preflight's existing rule-based signals into heuristic based risk scoring and deliver the Tier 2 pre-commit hook integration
- Adding measurable risk signal by defining the scoring logic using:
 1. File sensitivity patterns
 2. Label severity mapping
 3. Contributor intent keywords
- Add numerical risk score to advisory and make scoring configurable
Ex: $\text{risk_score} = (\text{auth_file_weight} \times \text{auth_score}) + (\text{label_severity_weight} \times \text{label_score}) + (\text{keyword_weight} \times \text{keyword_score})$

HIGH: $\text{score} > 0.7$

MEDIUM: $\text{score } 0.4 - 0.7$

LOW: $\text{score} < 0.4$

- Pre-Commit Hook (Tier 2):
 1. Implement ***preflight-hook*** Python entry point that reads staged files via ***git diff --cached --name-only*** and calls ***localhost:8000/analyze***
 2. Implement embedded offline fallback using rule-based signal mapping (no LLM) for when the advisory server is unavailable
 3. Author ***.pre-commit-config.yaml*** and register the hook with the pre-commit framework
 4. Configure hook exit behaviour based on severity and the `hook_mode` setting in `.preflight.yaml`
 5. Test the hook on 3-5 staged change scenarios: auth file modified, new API endpoint added, secrets pattern detected, docs-only change
- Share progress update and demo (hook firing in terminal) with mentors.

➤ Week 8 (July 12th to July 19th)

- Objective: Improvement of the Advisory UX of Preflight and begin the VS Code extension with a minimal maintainer dashboard
- Improve the formatting of the comments provided by the advisory
- Adding detailed modes and improving recommendation priority of the advisory

- Provide an example PR demonstration to the mentors with the polished advisory format
- VS Code Extension Foundation (Tier 1):
 1. Scaffold the TypeScript VS Code Extension using yo code
 2. Implement onSave handler that calls localhost:8000/analyze with the workspace's staged and modified file list
 3. Map the JSON advisory response to VS Code Diagnostic objects (ERROR for HIGH, WARNING for MEDIUM and INFORMATION for LOW)
 4. Display diagnostics in the problem panel and as gutter icons in the editor
 5. Add a VS Code status bar item showing the server connection state (connected/ server not running)
- Building a minimal dashboard UI for maintainers, the dashboard exposes data contract like Advisory log showing PR number, timestamp, severity level (HIGH/MEDIUM/LOW), OWASP category triggered, and whether the advisory was dismissed via preflight-inapplicable and False positive rate by counting dismissed advisories and total advisories also includes a configuration toggles.
- The dashboard reads from a lightweight SQLite log file written by the advisory pipeline on each PR event. It is built as a standalone module with no dependency on the core RAG pipeline, so it can be extended post-GSoC independently.
- Week 8 also serves as a buffer week if Week 7 pre-commit or heuristic scoring work runs over schedule. Must-have deliverables are the RAG system, heuristic scorer, pre-commit hook, and VS Code extension foundation. The dashboard is a secondary deliverable. Mentor check-ins are weekly via Slack with a written update every Sunday.
- Week 9 (July 19th to July 26th)
 - Objective: Complete the VS Code extension, Testing and finalizing the dashboard, expand OWASP documentation and refine heuristic scoring based on Week 7–8 results
 - VS Code Extension Completion:

1. Implement Quick Fix provider: clicking a diagnostic opens a side panel with the full advisory explanation and clickable OWASP references
 2. Add `preflight.serverUrl` configuration setting (default: <http://localhost:8000>) so teams can point the extension at a shared server.
 3. Package the extension as .vsix file for local installation; publish to the VS Code Marketplace if approved by mentors
 4. Test the extension on Ubuntu, macOS and Windows with both Docker and native server setups
- Expand OWASP documentation coverage by adding new sources to the vector store
 - Refine heuristic scoring weights based on false positive data collected in Weeks 7–8
 - Add a configurable YAML file allowing maintainers to adjust scoring thresholds without code changes ex:

```
preflight:
  enabled: true
  minimum_severity: MEDIUM
  similarity_threshold: 0.6
  chunk_size_tokens: 400
  chunk_overlap_tokens: 50
  heuristic_weights:
    auth_file_modified: 40
    security_label_present: 30
    new_api_endpoint: 25
    first_time_contributor: 20
    only_docs_changed: -30
  dismissal_label: preflight-inapplicable
  hook_mode: advisory      # 'advisory' or 'blocking'
  offline_fallback: true
  vscode_server_url: http://localhost:8000
```

- Test refined scoring on the existing evaluation suite and document improvements

➤ Week 10 (July 26th to August 2nd)

- Objective: End-to-end validation of all three tiers on all supported platforms

- Run full integration tests across all three tiers: VS Code extension, pre-commit hook and github actions workflow
- Validate the complete contributor workflow on Ubuntu 22.04, macOS 14 and Windows 11 (Docker path and native path)
- Run full evaluation of heuristic scorer: measure precision, false positive rate and stability
- Document cross-platform setup guide and troubleshooting steps
- Provide Evaluation report for mentors

➤ Week 11 (August 2nd to August 9th)

- Objective: Ensuring maintainability and technical documentation for all three tiers
- Prepare Architecture documentation covering the three-tier pipeline, Docker setup, VS Code extension installer and pre-commit hook usage
- Write a contributor quick-start guide: docker-compose up -d → pre-commit install → install VS Code extension → start contributing
- Refactor the codes for clarity and add inline documentation
- Deliver a complete technical documentation and clean production ready PR

➤ Week 12 (August 9th to August 16th)

- Final Term Evaluation
- Objective: Final stabilization and Demonstration of all three tiers
- Full integration tests and fix edge-case bugs if there any
- Prepare the final demo showcasing all three tiers: contributor receives guidance in VS Code, then at commit via the hook, then as a PR comment on Github and document the future roadmap and address mentor feedback

- Deliver a Stable RAG enhanced Preflight advisory system with Heuristic risk scoring with a minimal maintainer dashboard, VS Code extension, pre-commit hook, cross-platform Docker Setup and Documentation .

AI/LLM Usage Considerations:

- [] My GSoC proposal does not include any AI-generated content
- [✓] My GSoC proposal includes AI-generated content, as disclosed below:
 - AI Tools: ChatGPT, Claude, Gemini
 - LLMs and versions: GPT-4.1, Claude Sonnet 4.6, Gemini 2.5 Pro
 - Prompts: Used targeted prompts for grammar correction, clarity improvement, and concise rephrasing of technical content. Additional prompts were used to refine architectural explanations, validate scope feasibility, and identify unclear or ambiguous sections. No prompts were used to generate core system design or implementation logic.

AI/LLM tools were used for specific in-scope purposes for the proposal such as refining language clarity, brainstorming alternative phrasings, and reviewing grammar and formatting.

Also, LLMs were used for supportive or editorial purposes in this work. All outputs were reviewed and validated by the contributor to ensure accuracy and originality.

NOTE: All architectural decisions, system design choices and implementation plans were structured and reasoned by me based on my experience with BLT codebase, discussions within the community and my understanding of the Preflight pipeline.

During development, AI tools may be used for reference purposes such as API lookups, debugging assistance, or minor refactoring suggestions. However, all generated code will be manually reviewed, validated and tested before submission.

I take full ownership of the design, implementation, and security considerations of this project. AI tools will serve strictly as productivity aids, not autonomous decision-makers.

Pre-Selection Prototype:

This link consists a small demo for the GSoC Proposal :

<https://drive.google.com/file/d/1j7jYxn27RLytX15KhNckJlM6WLFgnn3f/view?usp=sharing>

The link to the repo : <https://github.com/S3DFX-CYBER/BLT-Preflight-Prototype>

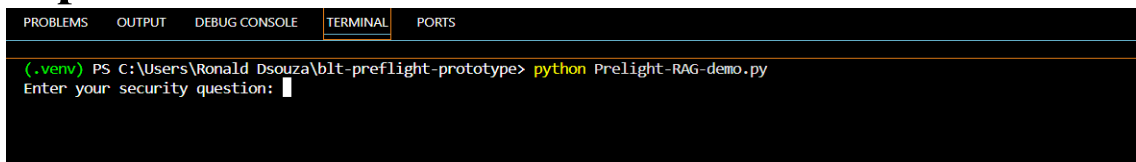
This repository contains a minimal offline Retrieval-Augmented Generation (RAG) prototype designed to demonstrate how security documentation can be queried using semantic search.

The goal of this demo is to illustrate how contributor tools like Preflight can provide contextual security guidance by retrieving relevant information from OWASP-style documentation.

This implementation is intentionally lightweight and runs fully offline, using embedding-based semantic retrieval powered by sentence-transformers and a vector store (ChromaDB) to fetch relevant context.

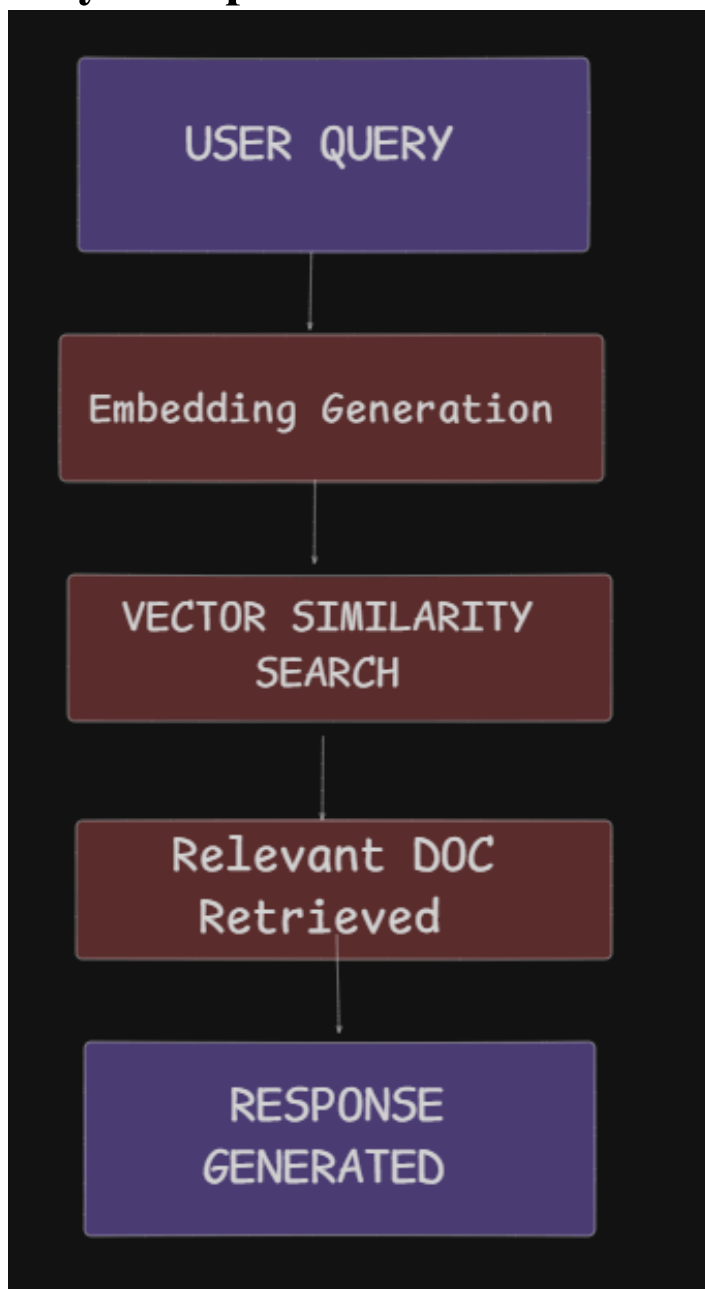
Example Proof of Concept Workflow:

1. Input:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\Users\Ronald Dsouza\blt-preflight-prototype> python Preflight-RAG-demo.py
Enter your security question: 
```

2. System processes:



3.Output:

Answer:
Based on retrieved OWASP context:
- Broken Access Control occurs when users can access resources they should not.
- Logging access control failures helps detect attacks.
- Rate limiting prevents automated attacks on access control.
- APIs without access control checks can be abused.
- Session management failures can let attackers act as other users.

Risks and Mitigation:

Sr, no	Name	Risk	Mitigation
1.	Inaccurate Retrieval from Documentation	The RAG system may retrieve irrelevant or incomplete information from OWASP documentation due to retrieval logic limitations or limited dataset coverage	Improve retrieval quality using better embedding models and vector similarity search. Expand and structure the dataset to ensure relevant security guidance is retrieved
2.	Hallucinated or Incorrect AI Responses	The LLM may generate responses not grounded in OWASP documentation, potentially giving incorrect security guidance	Use RAG so the model only answers based on retrieved context. Add prompt constraints to ensure answers rely strictly on provided documentation
3.	Dataset Limitations	The initial dataset may contain limited security examples, reducing the system's ability to answer diverse security questions	Expand the dataset using OWASP documentation, security guidelines, and curated vulnerability examples
4.	API Limitation	External AI APIs used for	Implement caching and modular architecture; use

	s or Rate Limits	embeddings or generation may have usage limits, downtime, or cost constraints	the defined fallback strategy (GPT-4o-mini → Ollama) if necessary
5.	Integration Complexity with Preflight	Integrating the RAG system with the existing Preflight workflow may require adjustments for compatibility	Develop the RAG module as a standalone FastAPI service with clear APIs for smooth integration into the Preflight ecosystem
6.	Performance and Latency Issues	Retrieval and model inference may increase response time when processing large documentation sets	Use efficient indexing and vector databases to optimize retrieval speed; pre-built ChromaDB index eliminates per-run embedding cost
7.	Alert Fatigue or False Positives	If the system over-flags PRs, contributors will start ignoring comments entirely, making the tool useless in practice	Implement confidence threshold gating: HIGH posts a comment requiring reviewer attention, MEDIUM posts an advisory-only comment, LOW is suppressed. Maintainers can label a PR preflight-inapplicable to dismiss irrelevant advisories
8.	Pre-Commit Hook Blocks Development Flow	If the hook is slow (LLM latency) or exits 1 unexpectedly, contributors will remove it, eliminating Tier 2 coverage	The hook has a configurable hook_mode (advisory vs blocking). The offline fallback ensures the hook completes in under 1 second when the server is unavailable. Default mode is advisory (never blocks), with blocking opt-in only

9.	VS Code Extension Causes IDE Slowdown	Triggering an HTTP call on every file save could slow down the editor, especially on large workspaces	Analysis is debounced (500ms idle post-save, as described in the VS Code Extension Architecture). If the server does not respond within 2 seconds, the extension silently skips and tries again on the next save
10.	Cross-Platform Docker Compatibility	Docker Desktop behaves differently on Windows (WSL2 backend) vs macOS vs Linux, potentially causing localhost networking issues	The docker-compose.yml uses host.docker.internal for cross-platform localhost resolution and is tested on all three platforms during Week 1. The native install path (setup.sh / setup.bat) is provided as a verified fallback

About Me:

I am a first-year student in my 2nd Semester pursuing a degree (Bachelor of Management Studies) at Mumbai University, India. I developed a significant interest in cybersecurity and programming during my first year of college.

I am always interested in learning curiously and exploring the depths of programming, cybersecurity and ai/ml. My Security learning journey began with hands-on labs and CTF platforms such as TryHackMe and Hack The Box, alongside participation in bug bounty programs on HackerOne. Throughout this journey, I regularly relied on resources from OWASP, particularly the OWASP Top 10 and API Security documentation.

My open-source journey began with contributions to OWASP VWAD, where I gained my first experience working within a structured code review process.

Currently, I am an active contributor to OWASP BLT, where in the past two months I have made over 200 contributions and had 20+ pull requests merged. Beyond code contributions, I have participated in roadmap discussions and architectural brainstorming, which has given me familiarity with the project's structure and long-term direction.

Alongside Open-Source work, I also enjoy building various experimental projects that intersect cybersecurity and AI-assisted workflows. Some of my projects include:

1. **[GSOC Organisation finder Tool:](#)** A website that is designed to help beginners identify suitable organisations from the official gsoc list based on their interests and tech stack.
2. **[TENET.AI \(WIP\):](#)** An experimental system exploring how autonomous AI agents could assist in cybersecurity. The Project is currently in progress.
3. **[Cybersecurity AI reporting agent:](#)** A N8N prototype security analysis pipeline that processes scan output such as NMAP or Raw vulnerability data) and generates structured security assessment reports with remediation guidance.

Through these projects, I have developed experience in Security Tools and automation , AI-assisted security workflows and designing experimental security prototypes , these interests directly align with the goals of the BLT Preflight project.

Motivation & Post-GSOC:

My motivation for working on this project comes from my growing involvement with the OWASP community and my experience contributing to the OWASP BLT project. Over the past months, I have actively contributed to BLT and learned how large open-source security projects operate in practice. Through these contributions, I became interested in improving developer security workflows and helping contributors understand potential security risks earlier in the development process.

BLT Preflight presents an exciting opportunity to combine modern AI-assisted techniques with practical security guidance. By building a RAG-enhanced advisory system, the project can help contributors receive contextual security feedback directly during pull request workflows. This aligns strongly with my interests in cybersecurity, secure development practices, and practical applications of machine learning in security tooling.

Beyond the technical implementation, this project is meaningful to me because it allows me to contribute something valuable to a community that has already played an important role in my learning journey. OWASP resources such as security documentation and testing guides have been a key part of my cybersecurity education, and contributing back to the ecosystem is something I value deeply.

Post-GSoC Commitment

My involvement with BLT and OWASP will not end after the GSoC program. I plan to continue contributing to the BLT ecosystem by maintaining and improving the Preflight advisory system, addressing

community feedback, and helping other contributors adopt and extend the tool. I will be available full-time for the project duration, as my academic semester concludes prior to the coding period.

System Specs:

OS: Windows 11

System type: x64

Processor: AMD Ryzen 5 5500U with Radeon Graphics

Contributions to BLT:

I started contributing to BLT Community in January 2026, below is a selection of my merged PRs in BLT repositories

NOTE: All PRS are merged

BLT-CORE

PR No.		Status
PR#5487	The PR fixes issue #5485 as it includes automated daily scheduler and syncing of leaderboard website data.	Merged
PR#5381	The PR fixes a security vulnerability CWE-295 with a one line fix.	Merged
PR#5390	The PR Fixes the check-peer-review workflow incorrectly runs on dependabot PRs	Merged
PR#5465	The PR fixes the github actions leaderboard of BLT counts only merged pr and discussed new features with donnie, rosai and added	Merged
PR#5544	The PR fixes the docker-tests job in the CI/CD workflow to checkout the PRs head commit instead of main branch	Merged

PR#5874	The PR includes an updated Security.md for OWASP BLT	Merged

BLT-LEAF

PR.No		Status
PR#147	The PR implements Github passive rate limiting tracking	Merged
PR#60	Added a CI workflow for validation checks for better code quality	Merged
PR#22	Added an automated setup script for simplification of local development for new contributors	Merged

BLT-IDEAS

PR.No		Status
PR#3	Add project E idea	Merged
PR#8	Update Project E idea	Merged
PR#11	Create Project E (extended)	Merged

A complete list of my contributions can be found here:

<https://github.com/pulls?q=is%3Apr+is%3Amerged+author%3AS3D-FX-CYBER+org%3AOWASP-BLT>

Contributions to OWASP:

I have also contributed to owasp projects:

[PR#235](#): Added an automated bot REPO-SCOUT for OWASP VWAD

I have also contributed to many owasp project on Security.md for OWASP VWAD, NETTACKER, WSTG, API SECURITY etc

Found a medium 5.4 Vulnerability at OWASP VWAD

GHSA: GHSA-ghh3-hgpg-p7cw

Community Recognition:



Rick Mitchell  · 1st

Application Security Professional - Known for community contributions and Open Source efforts
December 29, 2025, Rick worked with Savio but they were at different companies

Savio recently made a number of contributions to the OWASP Vulnerable Web Applications Directory project (<http://owasp.org/vwad>). He's been quick to point out and fix potential issue as well as undertaking new contributions. He's done a great job staying in-touch and addressing feedback as it was provided. Thanks for your help Savio!

Recognition

Security researchers and contributors who responsibly disclose vulnerabilities will be **publicly acknowledged** in our release notes (unless anonymity is requested).

- From our previous project repo: [Workflow Uses Untrusted Input in actions/checkout Ref Parameter](#) from [@S3DFX-CYBER](#).

References:

- <https://owasp.org/Top10/2025>
- <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/stable-en>
- https://owasp.org/www-community/Threat_Modeling
- https://owasp.org/www-community/Threat_Modeling_Process
- https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html
- https://huggingface.co/docs/transformers/en/model_doc/rag
- <https://huggingface.co/blog/ngxson/make-your-own-rag>
- <https://devguide.owasp.org/en/11-security-gap-analysis/02-bl>
- <https://code.visualstudio.com/api/get-started/your-first-extension>
- <https://code.visualstudio.com/api/language-extensions/programmatic-language-features#provide-diagnostics>
- <https://pre-commit.com/>
- <https://docs.docker.com/compose/>

----- THANK YOU -----